

NOIP 模拟赛

2025 年 11 月 23 日

题目名称	最小的大数	弹射起飞	旅行记录	古老卷轴
题目类型	传统型	传统型	传统型	传统型
可执行文件名	sbnun	catapult	trip	scroll
输入文件名	sbnun.in	catapult.in	trip.in	scroll.in
输出文件名	sbnun.out	catapult.out	trip.out	scroll.out
时间限制	1.0 秒	1.0 秒	3.0 秒	2.0 秒
内存限制	512 MB	512 MB	512 MB	512 MB
测试点数目	20	20	20	20

提交源程序文件名

对于 C++	sbnun.cpp	catapult.cpp	trip.cpp	scroll.cpp
--------	-----------	--------------	----------	------------

编译选项

对于 C++	-O2 -std=c++14
--------	----------------

注意事项（请仔细阅读）

1. 选手提交的源程序请放置在以题目对应文件名命名的子文件夹下。
2. 这套模拟赛的题目不一定按照难度顺序排列!!!
3. 文件名（包括程序名和输入输出文件名）必须使用英文小写。
4. C++ 中函数 main() 的返回值类型必须是 int，值必须为 0。
5. 对于因未遵守以上规则对成绩造成的影响，相关申诉不予受理。
6. 若无特殊说明，结果比较方式为忽略行末空格、文末回车后的全文比较。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。

最小的大数 (sbnun)

【题目描述】

小 P 在研究大整数。他在一排放置了 n 张卡片，每张卡片上写有一个从 1 到 9 的数字。连起来就是一个 n 位整数 s 。

一次操作中，小 P 可以将相邻的两张卡片交换，且最多可以执行 k 次这样的操作。

现在小 P 希望你求出：在不超过 k 次操作后，能得到的最小的 n 位数是多少？

【输入格式】

从文件 *sbnun.in* 中读入数据。

本题采用多组数据测试。

第一行包含一个整数 t ，表示测试数据的组数。接下来的 t 行每行描述一组测试数据。

每组测试数据用一行表示，包含一个数字字符串 s 和一个整数 k 。

【输出格式】

输出到文件 *sbnun.out* 中。

对于每个测试用例，输出一行，包含在不超过 k 次相邻交换后的最小可能的 n 位数。

【样例 1 输入】

```
4
321 0
9 1
21241127 10
692 1
```

【样例 1 输出】

```
321
9
11122247
629
```

【测试点约束】

对于 30% 的数据，满足 $\sum |s| \leq 8$ 。

对于 60% 的数据，满足 $\sum |s| \leq 1000$ 。

对于全部数据，满足 $1 \leq |s| \leq 10^5$ ， $\sum |s| \leq 10^5$ ， $0 \leq k \leq 10^{18}$ ， $1 \leq t \leq 10^5$ ， s 是一个仅由数字 1 到 9 组成的正整数。

弹射起飞 (catapult)

【题目描述】

G 国发明了一种革命性的全新交通系统。这一系统不需要道路，也不依赖复杂的机械结构，只依靠巨大的弹射器进行飞行运输。

G 国共有 n 座城市，每座城市都安装有一台弹射器。每当有人要出行时，他会进入一个特殊的胶囊舱，然后弹射器会将胶囊发射到另一座城市。每台弹射器的威力足以将胶囊投掷至任意其他城市，并且可同时承载任意数量的乘客。

然而，问题在于每台弹射器充能时间极长，因此每天只能使用一次。

某位乘客可能需要多次使用弹射器才能到达目的地。例如，若乘客希望从城市 A 前往城市 B ，他可以先从 A 弹射到城市 C ，再由 C 弹射至 B 。

弹射器发射按顺序进行，每次发射时，位于该城市的乘客（包括之前被发射到该城市的乘客）都可以被发射到目标城市。

今天共有 m 名乘客，第 i 名乘客希望从城市 a_i 前往城市 b_i 。你的任务是设计一个在同一天内完成所有乘客运输的方案，使得所使用的弹射器总数最少；若无法完成，则输出 -1 。

【输入格式】

从文件 `catapult.in` 中读入数据。

第一行包含两个整数 n 和 m ，分别表示城市数量与乘客数量。

接下来的 m 行中，每行包含两个整数 a_i 和 b_i ，表示第 i 名乘客希望从城市 a_i 前往城市 b_i 。

【输出格式】

输出到文件 `catapult.out` 中。

如果可以在一天内使所有乘客到达目的地，请在第一行输出一个整数 k 表示需要使用的最少弹射器数量。

接下来输出 k 行，每行包含两个整数 c_i 和 d_i ，表示依次从城市 c_i 发射弹射到城市 d_i ，保证 $c_i \neq d_i$ ，所有 c_i 互不相同。

注意，你无需输出每次发射时应将哪些乘客放入胶囊，只需保证存在一种方案使得所有乘客最终都能到达目的地即可。如果有多种满足要求的方案，输出任意一个即可。

若无法完成所有乘客的运输，请输出单独一行数字 -1 。

【样例 1 输入】

```
5 6
1 3
1 2
2 3
4 2
1 5
5 1
```

【样例 1 输出】

```
5
5 4
4 1
1 2
2 3
3 5
```

【样例 2/3】

见下发文件。

【测试点约束】

对于 30% 的数据，满足 $n \leq 5$, $m \leq 10$ 。

对于 50% 的数据，满足 $n \leq 100$, $m \leq 1000$ 。

对于另外 30% 的数据，满足不存在 $p_0, p_1, \dots, p_k$ 使得 $\forall i \in [0, k], \exists j \in [1, m]$ 有 $p_i = a_j, p_{(i+1) \bmod k+1} = b_j$ 。

对于全部数据，满足 $n \leq 1000$, $m \leq 10^5$ 。

旅行记录 (trip)

【题目描述】

在远古时期，G 国中共有 n 个城市，而每个城市则各自有一个风险度 w ，且满足 $w_1 \leq w_2 \leq \dots \leq w_n$ 。

G 国的居民们都很喜欢旅行，不过 G 国的城市之间有一个规定：如果一个旅客从城市 i 前往城市 j ，如果出发城市的风险度过高则需要一段时间来降低风险。具体的，若 $w_i > w_j + m$ ，那么该旅客在抵达城市 j 后需要停留 t_j 天来降低风险度，随后才能继续旅行。反之则无需停留。

由于 G 国的交通非常方便，任意两个城市之间都可以直接通行，且每次跨城旅行耗时 1 天。

目前有 q 名旅客的行程记录。对于每名旅客，已知其出发城市和最终目的城市。请计算每名旅客完成从出发城市到目的城市的最短可能旅行时间。旅行可以经过若干中转城市，每次移动需耗时 1 天，若入城时风险度过高则还需额外增加相应的天数，**包括目的地城市**。

【输入格式】

从文件 **trip.in** 中读入数据。

第一行包含三个整数 n, q, m 。

第二行包含 n 个整数 $w_1, \dots, w_n$ 。

第三行包含 n 个整数 $t_1, \dots, t_n$ 。

接下来的 q 行，每行包含两个整数 x 和 y ，表示一名旅客从城市 x 出发、最终目的地为城市 y 。

【输出格式】

输出到文件 **trip.out** 中。

对于每名旅客，输出一行一个整数，表示从出发城市到目的城市的最短需要花费多少天。

【样例 1 输入】

```
5 4 10
0 8 20 25 30
5 11 13 6 3
```

```
5 1
5 2
5 3
5 4
```

【样例 1 输出】

```
6
7
1
1
```

【样例 2/3】

见下发文件。

【测试点约束】

对于 40% 的数据，满足 $n, q \leq 1000$ 。

对于另外 20% 的数据，满足 $t_i = 10^9$ ， $w_{i+1} - w_i < m$ 。

对于 80% 的数据，满足 $n, q \leq 10^5$ （包含上一档特殊数据）。

对于全部数据，满足 $2 \leq n \leq 10^6$ ， $1 \leq q \leq 10^6$ ， $0 \leq m \leq 10^9$ ， $0 \leq w_1 \leq w_2 \leq \dots \leq w_n \leq 10^9$ ， $0 \leq t_i \leq 10^9$ ， $1 \leq x, y \leq n$ ， $x \neq y$ 。

古老卷轴 (scroll)

【题目描述】

在一座古老的图书塔中，小 P 正在整理一批神奇的卷轴。这些卷轴依次编号为 1 到 $n \times m$ ，并以某种方式被重新排列成一个长度为 $n \times m$ 的序列。小 P 称一个排列为**和谐排列**，当且仅当：

- 该排列的最长上升子序列长度恰好为 n 。
- 该排列的最长下降子序列长度恰好为 m 。

现在小 P 想进一步研究**和谐排列**的结构。他会钦定某个位置 a 上必须放卷轴 b ，记这样的限制下和谐排列的数量为 $f(a, b)$ 。请你对所有的 $1 \leq a, b \leq n \times m$ 求出 $f(a, b)$ 。

由于答案很大，你只需要输出答案对 mod 取模后的结果。

【输入格式】

从文件 **scroll.in** 中读入数据。

第一行三个整数表示 n, m, mod 。

【输出格式】

输出到文件 **scroll.out** 中。

输出 $n \times m$ 行，每行包含 $n \times m$ 个整数。第 i 行 j 列表示 $f(i, j)$ 对 mod 取模后的值。

【样例 1 输入】

```
3 2 571901821
```

【样例 1 输出】

```
0 10 10 5 0 0
10 0 0 6 9 0
10 0 0 4 6 5
5 6 4 0 0 10
0 9 6 0 0 10
0 0 5 10 10 0
```


【样例 2】

见下发文件。

【测试点约束】

对于 15% 的数据，满足 $n \times m \leq 10$ 。

对于 30% 的数据，满足 $n \times m \leq 20$ 。

对于 50% 的数据，满足 $n \times m \leq 50$ 。

对于另外 20% 的数据，满足 $\text{mod} = 998244353$ 。

对于全部数据，满足 $1 \leq n \times m \leq 100$ ， $10^8 \leq \text{mod} \leq 10^9$ ， mod 为质数。